

Attendance Management System Documentation

Overview

This Attendance Management System is built using Spring Boot and PostgreSQL, designed to handle various types of events such as exams, meetings, and student events. The system allows administrators to configure event-specific requirements, participants to mark attendance securely, and ensures security through geolocation and IP-based validations.

Modules

1. Admin Authentication and Management

Purpose:

To provide secure access for administrators to manage events and configure attendance modules.

Key Features:

- Admin registration and login functionality.
- Password encryption using `BCryptPasswordEncoder`.
- Role-based access control using Spring Security.

APIs:

1. Admin Registration

- **Endpoint:** `/admin_register`
- **Method:** `POST`
- **Request Payload:**

```
{
  "username": "admin1",
  "password": "securepassword",
```

```
"email": "admin@example.com"
}
```

- **Response:**

```
{
  "message": "Admin registered successfully."
}
```

2. Admin Login

- **Endpoint:** `/admin_login`

- **Method:** `POST`

- **Request Payload:**

```
{
  "username": "admin1",
  "password": "securepassword"
}
```

- **Response:**

```
{
  "message": "Login successful."
}
```

2. Event Creation and Management

Purpose:

To allow administrators to create and manage events with custom requirements.

Key Features:

- Customizable fields for each event.
- Validation of event status (e.g., active or inactive).

APIs:

1. Create Event

- **Endpoint:** `/event/create`
- **Method:** `POST`
- **Request Payload:**

```
{
  "eventName": "Annual Meeting",
  "eventType": "Meeting",
  "startTime": "2025-01-10T09:00:00",
  "endTime": "2025-01-10T12:00:00",
  "locationCoordinates": {
    "longitude": 77.5946,
    "latitude": 12.9716
  },
  "locationRange": 100
}
```

- **Response:**

```
{
  "eventId": "e12345",
  "message": "Event created successfully."
}
```

3. Participant Management

Purpose:

To handle participant data for events, including bulk upload via CSV or Excel.

Key Features:

- Bulk participant insertion using file upload.
- Validation of participant data against event-specific requirements.

APIs:

1. Bulk Upload Participants

- **Endpoint:** `/participants/upload`
- **Method:** `POST`
- **Request Payload:**
Form-data with a file upload field named `file`.
- **Response:**

```
{  
  "message": "Participants uploaded successfully."  
}
```

4. Attendance Marking

Purpose:

To allow participants to mark their attendance securely based on geolocation and event-specific validations.

Key Features:

- Geolocation-based proximity validation.
- IP blocking to prevent duplicate attendance.

APIs:

1. Mark Attendance

- **Endpoint:** `/attendance/mark`
- **Method:** `POST`
- **Request Payload:**

```
{  
  "participantId": "REG12345",  
  "eventId": "e12345",  
  "coordinates": {  
    "longitude": 77.5946,  
    "latitude": 12.9716  
  }  
}
```

```
}  
}
```

- **Response:**

```
{  
  "message": "Attendance marked successfully."  
}
```

5. Security Configurations

Purpose:

To ensure secure access to APIs and resources.

Key Features:

- CSRF protection (disabled for simplicity in this iteration).
- Password encryption for admin accounts.

Configuration:

Spring Security is configured in `SecurityConfig` to restrict access to sensitive endpoints and allow open access to public endpoints like login and registration.

6. Database Design (PostgreSQL)

Tables:

1. `admins`

- Stores admin credentials and profile information.
- Columns: `id`, `username`, `password`, `email`.

2. `events`

- Stores event details.
- Columns: `id`, `name`, `type`, `start_time`, `end_time`, `longitude`, `latitude`, `range`, `status`.

3. `participants`

- Stores participant information for events.
- Columns: `id` , `event_id` , `name` , `registration_number` , `email` , `phone` .

4. `attendance`

- Stores attendance records.
- Columns: `id` , `participant_id` , `event_id` , `status` , `timestamp` .

5. `blocked_ips`

- Tracks blocked IPs to prevent duplicate attendance.
 - Columns: `id` , `ip_address` , `event_id` .
-

7. Future Improvements

- Allow admins to specify the participant identification field during event creation.
 - Implement advanced reporting features with filters and analytics.
 - Enhance security by enabling CSRF protection and rate limiting.
-

Testing Notes

Postman test cases have been provided for:

- Admin registration and login.
- Event creation and retrieval.
- Participant bulk upload.
- Attendance marking.

Thorough testing for edge cases (e.g., invalid inputs, unauthorized access) is recommended.